

Object Oriented Systems (OOS) — CO3 Complete Detailed Solutions

Question 3 (a), (b), (c), and (d)

Based on the uploaded university examination paper.

Introduction

Question 3 of CO3 focuses on some of the most advanced and powerful features of Java Programming:

-  Generic Programming
-  Reflection (Introspection)
-  Generic Methods
-  String Class Manipulation
-  Dynamic Method Invocation

These topics are extremely important because modern Java applications require reusable code, type safety, runtime flexibility, metadata inspection, and dynamic execution. Java achieves these capabilities using:

-  Generics
-  Reflection API
-  Runtime Class Loading
-  Dynamic Binding

This entire question represents the transition from basic object-oriented programming to advanced enterprise-level Java programming.  

In this answer, every concept is explained deeply with theory-rich explanation, JVM/runtime analysis, detailed Java code, output explanation, conceptual understanding, and real-world intuition.

Question 3(a)

Exact Question

Define a generic class with a member method `frequency(A)` that takes arrays `A` of different datatypes (int, float, char) as input and finds the frequency of each element. **OR** Define a generic class with member method `BinSearch(A, P)` that takes a sorted array `A` of different datatypes (int, char, float, double) and an element `P` and finds whether `P` is present using Binary Search. Also find the location of the element.

Part 1 — Generic Frequency Counter

 **Introduction to Generics** Before Java Generics were introduced, collections and methods accepted objects of type `Object`. This created major problems:

-  unsafe type conversions
-  runtime `ClassCastException`

- ⚠️ less reusable code

Generics were introduced in Java 5 to solve these issues.

 **What Are Generics?** Generics allow:

- ✅ type-safe programming
- ✅ reusable algorithms
- ✅ compile-time checking
- ✅ elimination of explicit casting

Generic Class Syntax

```
class Demo<T> {  
}
```

Here, **T** → **Type Parameter**. It represents any datatype.

 **Problem Objective** We must create a generic class that accepts arrays of different datatypes, calculates the frequency of elements, and works for Integer, Float, and Character.

Java Program

```
import java.util.*;  
  
class Frequency<T> {  
  
    void frequency(T arr[]) {  
        HashMap<T, Integer> map = new HashMap<>();  
  
        for(T element : arr) {  
            if(map.containsKey(element)) {  
                map.put(element, map.get(element) + 1);  
            } else {  
                map.put(element, 1);  
            }  
        }  
  
        System.out.println("Frequency of Elements:");  
        for(Map.Entry<T, Integer> e : map.entrySet()) {  
            System.out.println(e.getKey() + " -> " + e.getValue());  
        }  
    }  
}  
  
class Demo {  
    public static void main(String args[]) {  
        Frequency<Integer> obj1 = new Frequency<>();  
        Integer arr1[] = {1, 2, 1, 3, 2, 1, 4};  
        obj1.frequency(arr1);  
    }  
}
```

```
Frequency<Character> obj2 = new Frequency<>();
Character arr2[] = {'a', 'b', 'a', 'c', 'b'};
obj2.frequency(arr2);
    }
}
```

Output

```
Frequency of Elements:
1 -> 3
2 -> 2
3 -> 1
4 -> 1
Frequency of Elements:
a -> 2
b -> 2
c -> 1
```

 **Why This Output Occurs** The `HashMap` stores **Key** (Element) and **Value** (Frequency). Whenever an element repeats, `map.put(element, map.get(element) + 1);` updates its frequency.

Deep Conceptual Analysis

- **Why Generics Are Important:** Without Generics, `Object arr[]` would require manual type conversion. Generics provide compile-time safety, cleaner syntax, and reusable algorithms.
- **JVM Internal Behavior:** Java Generics use  **Type Erasure**. During compilation, `Frequency<Integer>` becomes `Frequency<Object>`. The JVM removes generic type information to maintain backward compatibility.

 **Real-World Intuition** Imagine a frequency machine. Instead of building separate machines for numbers, letters, and floating values, one intelligent generic machine handles everything. 

Advantages & Disadvantages

Advantage	Explanation
Reusable	One code works for many types
Safe	Prevents invalid types
Maintainable	Reduces duplicate code
Efficient	Compile-time checking
Disadvantage	Explanation
Type Erasure	Generic info removed at runtime
Primitive Limitation	Requires wrapper classes

❏ **Conclusion** Generic programming makes Java flexible, reusable, and type-safe for modern software systems.

🌟 Part 2 — Generic Binary Search

■ **Introduction** Binary Search is one of the most efficient searching algorithms. Instead of checking elements sequentially, it divides the array repeatedly, reduces the search space, and achieves logarithmic complexity.

■ Time Complexity

Case	Complexity
Best	$O(1)$
Average	$O(\log n)$
Worst	$O(\log n)$

💡 **Core Idea** Binary Search requires a sorted array, middle element comparison, and repeated partitioning.

💻 Java Program

```
class BinSearch<T Comparable<T extends>> {

    void search(T arr[], T key) {
        int low = 0;
        int high = arr.length - 1;
        boolean found = false;

        while(low <= high) {
            int mid = (low + high) / 2;
            int result = key.compareTo(arr[mid]);

            if(result == 0) {
                System.out.println("Element Found at Position: " + mid);
                found = true;
                break;
            } else if(result > 0) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }

        if(!found) {
            System.out.println("Element Not Found");
        }
    }
}

class Demo {
    public static void main(String args[]) {
```

```
        BinSearch<Integer> obj = new BinSearch<>();
        Integer arr[] = {10, 20, 30, 40, 50};
        obj.search(arr, 40);
    }
}
```

Output

Element Found at Position: 3

 **Why This Output Occurs** Search process:

Step	low	high	mid	Value
1	0	4	2	30
2	3	4	3	40

Element found at index 3.

 **Why Comparable Is Used** `<T extends Comparable<T>>` ensures objects can be compared safely. Without it, comparison using `>`, `<`, or `==` on objects is impossible.

 **Conclusion** Generic Binary Search combines efficiency and reusability in one elegant implementation.

Question 3(b)

Exact Question

Show how a static generic method is defined in a class and how it is invoked.

 **Introduction** A generic method allows independent type parameters inside methods. When combined with `static`, the method becomes class-level, object creation is unnecessary, and reusable utility functions become possible.

Syntax

```
static <T> void method(T data)
```

Java Program

```
class Demo {

    static <T> void display(T value) {
        System.out.println("Value = " + value);
    }
}
```

```

    public static void main(String args[]) {
        Demo.display(100);
        Demo.display("Java");
        Demo.display(5.67);
    }
}

```

Output

```

Value = 100
Value = Java
Value = 5.67

```

 **Why This Output Occurs** The compiler automatically determines the datatype:

- 100 -> Integer
- "Java" -> String
- 5.67 -> Double

 **JVM Internal Working** Generics undergo type erasure. Separate versions are not created; the compiler ensures type safety before runtime.

 **Real-World Intuition** A static generic method behaves like a universal utility tool. It adapts automatically depending on the input.

 **Conclusion** Static generic methods provide reusable type-safe utilities without requiring object creation.

Question 3(c)

Exact Question

Write suitable code snippet to invoke the: `startsWith()`, `equalsIgnoreCase()`, `toUpperCase()`, `concat()`, `trim()`, `compareTo()`, `lastIndexOf()` methods of String class using Reflection. Also invoke overloaded versions of `substring()` using Reflection.

 **Introduction to Reflection** Reflection is one of the most advanced capabilities of Java.  It allows a program to inspect classes dynamically, invoke methods at runtime, access metadata, and create objects dynamically. This capability is heavily used in the Spring Framework, Hibernate, IDEs, Testing frameworks, and Dependency Injection systems.

 **What Is Reflection?** Reflection means a Java program examining and manipulating itself during runtime.

Reflection API Classes

Class	Purpose
Class	Represents class metadata

Class	Purpose
Method	Represents methods
Constructor	Represents constructors
Field	Represents variables

Java Program

```
import java.lang.reflect.*;

class Demo {
    public static void main(String args[]) throws Exception {
        String str = " helloJava ";
        Class c = str.getClass();

        Method m1 = c.getMethod("startsWith", String.class);
        System.out.println(m1.invoke(str, " he"));

        Method m2 = c.getMethod("equalsIgnoreCase", String.class);
        System.out.println(m2.invoke(str, " HELLOJAVA "));

        Method m3 = c.getMethod("toUpperCase");
        System.out.println(m3.invoke(str));

        Method m4 = c.getMethod("concat", String.class);
        System.out.println(m4.invoke(str, " WORLD"));

        Method m5 = c.getMethod("trim");
        System.out.println(m5.invoke(str));

        Method m6 = c.getMethod("compareTo", String.class);
        System.out.println(m6.invoke(str, "abc"));

        Method m7 = c.getMethod("lastIndexOf", String.class);
        System.out.println(m7.invoke(str, "a"));

        Method sub1 = c.getMethod("substring", int.class);
        System.out.println(sub1.invoke(str, 2));

        Method sub2 = c.getMethod("substring", int.class, int.class);
        System.out.println(sub2.invoke(str, 2, 7));
    }
}
```

Output

```
true
true
HELLOJAVA
```

```
helloJava    WORLD
helloJava
-65
7
helloJava
hello
```

 **Why This Output Occurs** Reflection dynamically locates methods using `getMethod()` and then invokes them using `invoke()`.

Deep Reflection Analysis

- **Dynamic Method Invocation:** Normally `str.trim()`; is statically bound. Reflection allows runtime resolution.
- **JVM Internal Behavior:** During reflection, the JVM loads metadata tables, searches method signatures, performs access checks, and dynamically invokes methods. Reflection is slower because runtime lookup occurs.

 **Real-World Intuition** Reflection is like a robot mechanic inspecting machine parts dynamically without prior knowledge.

 **Conclusion** Reflection enables highly dynamic and intelligent Java applications by supporting runtime inspection and invocation.

Question 3(d)

Exact Question

Discuss the major drawbacks of Reflection with suitable code snippets.

 **Introduction** Although Reflection is extremely powerful, it comes with serious drawbacks. Reflection bypasses many compile-time safety mechanisms.

Major Drawbacks of Reflection

 **1 Performance Overhead** Reflection is slower than normal method invocation.

```
Method m = obj.getClass().getMethod("show");
m.invoke(obj);
```

Reason: JVM performs runtime lookup, access verification, and dynamic dispatch. This increases execution time.

 **2 Security Problems** Reflection can access private members.

```
Field f = obj.getClass().getDeclaredField("x");  
f.setAccessible(true);
```

Risk: Sensitive data may become exposed, violating encapsulation.

✨ **3 Loss of Compile-Time Checking** Method names are passed as strings.

```
getMethod("shoow") // Typo won't be caught by compiler
```

Risk: Typographical errors are detected only during runtime (throws `NoSuchMethodException`).

✨ **4 Complex Debugging** Reflection-based code is harder to understand and maintain.

✨ **5 Breaks Encapsulation** Private members become accessible, which violates strict OOP principles.

📊 Summary Table

Drawback	Effect
Performance	Slower execution
Security	Private access possible
Runtime Errors	No compile-time safety
Complexity	Hard debugging
Encapsulation	OOP principles weakened

💡 **Real-World Intuition** Reflection is like a master key 🔑. It provides powerful access, but if misused, it can create security and maintenance problems.

🔗 **Final Conclusion** Question 3 demonstrates advanced Java programming concepts involving Generic Programming, Type Safety, Binary Search using Generics, Reflection API, Runtime Method Invocation, and Dynamic Programming Features. These features make Java one of the most flexible, reusable, and enterprise-ready programming languages in the software industry. 🍵🚀🔥